



05 토큰화

- 자연어(Natural Language) : 사람들이 쓰는 언어 활동을 위해 자연스럽게 만들어진 언어
- **자연어 처리(Natural Language Processing, NLP)** : 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술

→ 인공지능의 하위 분야 중 하나

→ 컴퓨터가 인간과 유사한 방식으로 인간의 언어를 이해하고 처리하는 것이 주요 목표

→ 인간 언어의 구조, 의미, 맥락을 분석하고 이해할 수 있는 알고리즘, 모델을 개발

- 모호성(Ambiguity): 인간의 언어는 단어와 구가 사용되는 맥락에 따라 여러 의미를 갖게 되어 모호한 경우가 많다. 알고리즘은 이러한 다양한 의미를 이해하고 명확하게 구분할 수 있어야 한다.
- 가변성(Variability): 인간의 언어는 다양한 사투리(Dialects), 강세(Accent), 신조어(Coined word), 작문 스타일로 인해 매우 가변적이다. 알고리즘은 이러한 가변성을 처리할 수 있어야 하며 사용 중인 언어를 이해할 수 있어야 한다.
- 구조(Structure): 인간의 언어는 문장이나 구의 의미를 이해할 때 구문(Syntactic)을 파악하여 의미(Semantic)를 해석한다. 알고리즘은 문장의 구조와 문법적 요소를 이해하여 의미를 추론하거나 분석할 수 있어야 한다.

⇒ 위의 세 가지 문제를 해결!

- **말뭉치(Corpus)**를 일정한 단위인 **토큰(Token)**으로 나뉘어야 함

말뭉치(Corpus)

- 자연어 모델을 훈련 & 평가하는 데 사용되는 대규모의 자연어
- 뉴스 기사, 사용자 리뷰, 저널이나 칼럼 등에서 목적에 따라 구축되는 텍스트 데이터
- 언어 모델 구축 & 평가하는 데 자주 사용됨

토큰(Token)

- 개별 단어나 문장 부호와 같은 텍스트 의미
- 말뭉치보다 더 작은 단위
- Ex. 텍스트의 개별 단어, 구두점, 기타 의미 단위

토큰화(Tokenization)

- 컴퓨터가 자연어를 이해할 수 있게 나누는 과정
- 컴퓨터가 텍스트를 효율적으로 분석 & 처리할 수 있게 하는 과정

토크나이저(Tokenizer)

- 텍스트 문자열을 토큰으로 나누는 알고리즘 or 소프트웨어

Ex)

- 입력: '형태소 분석기를 이용해 간단하게 토큰화할 수 있다.'
- 결과: ['형태소', '분석기', '를', '이용', '하', '어', '간단', '하', '게', '토큰', '화', '하', 'ㄹ', '수', '있', '다', '.']

[토크나이저 구축 방법]

- 공백 분할: 텍스트를 공백 단위로 분리해 개별 단어로 토큰화한다.
- 정규표현식 적용: 정규 표현식으로 특정 패턴을 식별해 텍스트를 분할한다.
- 어휘 사전(Vocabulary) 적용: 사전에 정의된 단어 집합을 토큰으로 사용한다.
- 머신러닝 활용: 데이터셋을 기반으로 토큰화하는 방법을 학습한 머신러닝을 적용한다.

어휘 사전(Vocabulary, Vocab)

- 사전에 정의된 단어를 활용해 토크나이저 구축하는 방법

- **OOV(Out of Vocab)**

: 어휘 사전에 없는 단어 or 토큰

- OOV 문제를 고려해 토큰화 해야함!
- But, 큰 어휘 사전 구축일수록 학습 비용 증대, 차원의 저주에 빠질 수 O!

단어 및 글자 토큰화

- **토큰화** - 텍스트 데이터를 구조적으로 분해하여 개별 토큰으로 나누는 작업 → 정확한 분석을 위해 필수, 단어나 문장의 빈도수, 출현 패턴 파악 가능
- 작은 단위로 분해된 텍스트 데이터 - 컴퓨터가 이해하고 처리하기 용이 → 기계 번역, 문서 분류, 감성 분석 등 다양한 자연어 처리 작업에 활용 가능

단어 토큰화(Word Tokenization)

- 텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업
- 띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용해 토큰화 수행
- split 매서드를 이용해 토큰화
- 구분자가 없다면 → 공백 기준으로 나누기

글자 토큰화(Character Tokenization)

- 띄어쓰기, 글자 단위로 문장 나누는 방식
- 작은 단어 사전 구축 가능
- 컴퓨터 자원 절약, 각 단어 학습 빈도수 증가
- 시퀀스 예측 작업(언어 모델링)에서 활용 가능
- 리스트 형태로 변환하여 수행
- 자소/자모 토큰화
- But, 문장 생성이나 개체명 인식의 경우에 다의어나 동음이의어가 많은 도메인에서 구별 어려울 수 O
- 시퀀스 길어질수록 연산량 증가

형태소 토큰화(Morpheme Tokenization)

- 텍스트를 형태소 단위로 나누는 토큰화 방법
- 언어의 문법 & 구조를 고려해 단어 분리, 의미 있는 단위로 분류
- 교착어 언어에서 중요하게 수행

***형태소** : 의미를 가진 최소의 단위

→ 형태소 분석을 통해 문장 내 각 형태소의 역할 파악 가능

→ 문장 이해 & 처리 가능

형태소 어휘 사전(Morpheme Vocabulary)

- 각 단어의 형태소 정보를 포함하는 사전
- 단어가 어떤 형태소들의 조합으로 이루어져 있는지 확인 가능
- 각 형태소가 어떤 품사에 속하는지, 해당 품사의 뜻 제공

***품사 태깅(POS Tagging)** : 각 형태소의 품사를 태깅하는 작업

→ 문맥 고려 가능 → 정확한 분석!

KoNLPy

- 한국어 자연어 처리를 위해 개발된 라이브러리
- 명사 추출, 형태소 분석, 품사 태깅 기능 제공
- 텍스트 데이터 전처리, 분석을 위한 도구, 함수 제공
- Okt(Open Korean Text), 꼬꼬마(Kkma), 코모란(Komoran), 한나눔(Hannanum), 메캅(Mecab) 등의 형태소 분석기 존재
- Okt 품사 태그

태그	품사	태그	품사
Noun	명사	Punctuation	구두점
Verb	동사	Foreign	외국어
Adjective	형용사	Alpha	알파벳
Determiner	관형사	Number	숫자
Adverb	부사	KoreanParticle	자음/모음
Conjunction	접속사	URL	웹 주소
Exclamation	감탄사	Hashtag	해시태그(#)
Josa	조사	Email	이메일
PreEomi	선어말어미	ScreenName	아이디 표기(@)
Eomi	어미	Unknown	미등록
Suffix	접미사		

- 꼬꼬마 품사 태그

태그	품사	태그	품사
NNG	보통명사	EPH	존칭 선어말 어미
NNP	고유명사	EPT	시제 선어말 어미
NNB	일반 의존 명사	EPP	공손 선어말 어미
NNM	단위 의존 명사	EFN	평서형 종결 어미
NR	수사	EFQ	의문형 종결 어미
NP	대명사	EFO	명령형 종결 어미
VV	동사	EFA	청유형 종결 어미
VA	형용사	EFI	감탄형 종결 어미
VXV	보조 동사	EFR	존칭형 종결 어미
VXA	보조 형용사	ECE	대등 연결 어미
VCP	강정 지칭사	ECS	보조적 연결 어미
VCN	부정 지칭사	ECD	의존적 연결 어미
MDN	수 관형사	ETN	명사형 전성 어미
MDT	일반 관형사	ETD	관형형 전성 어미

태그	품사	태그	품사
MAG	일반 부사	XPN	체언 접두사
MAC	접속 부사	XPV	용언 접두사
JKS	주격 조사	XSN	명사파생 접미사
JKC	보격 조사	XSV	동사 파생 접미사
JKG	관형격 조사	XSA	형용사 파생 접미사
JKO	목적격 조사	XR	어근
JKM	부사격 조사	SF	마침표, 물음표, 느낌표
JKI	호격 조사	SE	줄임표
JKQ	인용격 조사	SS	따옴표, 괄호표, 줄표
JC	접속 조사	SP	쉼표, 기온대점, 콜론, 빗금
JX	보조사	S0	붙임표(물결, 숨김, 빠짐)
OH	한자	SW	기타기호(논리수학기호, 화폐기호)
OL	외국어	IC	감탄사
ON	숫자	UN	명사추정범주

NLTK(Natural Language Toolkit)

- 자연어 처리를 위해 개발된 라이브러리
- 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등의 기능 제공
- 네덜란드어, 프랑스어, 독일어 등의 다양한 언어의 자연어 처리를 위한 데이터와 모델 제공
- Average Perceptron Tagger 품사 태그

태그	품사	태그	품사
CC	접속사	PDT	관사
CD	기수(Cardinal Number)	POS	소유격 조사
DT	관형사	PRP	인칭 대명사
EX	there의 대명사형태	PRP\$	소유격 대명사
FW	외래어	RB	부사

태그	품사	태그	품사
JN	전치사	RBR	비교급 부사
JJ	형용사	RBS	최상급 부사
JJR	비교급 형용사	RP	전치사 또는 조동사
JJS	최상급 형용사	VB	기본형 동사
LS	목록 표시	VBD	과거형 동사
MD	조동사	VBG	현재 부사형 동사
NN	단수형 명사	VBN	과거 분사형 동사
NNS	복수형 명사	VBP	1인칭 단수 현재형 동사
NNP	단수형 고유 명사와 서수(Ordinal Number)	VBZ	3인칭 단수 현재형 동사
NNPS	복수형 고유 명사	WDT	관계사
SYM	기호	WP	주격 관계대명사
TO	To 부정사	WP\$	소유격 관계대명사
UH	감탄사	WRB	관계부사

spaCy

- 사이썬(Cython) 기반으로 개발된 오픈 소스 라이브러리
- NLTK 라이브러리와 마찬가지로 자연어 처리를 위한 기능 제공
- 빠른 속도 & 높은 정확도 ← 머신러닝 기반의 자연어 처리 라이브러리
- spaCy 품사 태그

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
ADJ	JJ	형용사	NUM	CD	기수(Cardinal Number)
ADJ	JJR	비교급 형용사	PROPN	NNP	고유 명사
ADJ	JJS	최상급 형용사	PROPN	NNPS	복수형 고유 명사
ADP	IN	전치사	PUNCT	-LRB-	왼쪽 괄호
ADV	RB	부사	PUNCT	-RRB-	오른쪽 괄호
ADV	RBR	비교급 부사	PUNCT	,	쉼표
ADV	RBS	최상급 부사	PUNCT	:	콜론
ADV	WRB	관계 부사	PUNCT	.	마침표, 물음표, 느낌표
AUX	MD	조동사	PUNCT	...	생략 부호
CCONJ	CC	접속사	PUNCT	"	여는 따옴표
DET	DT	관사, 한정사	PUNCT	"	닫는 따옴표
DET	PDT	전치사 한정사	PUNCT	HYPH	하이픈
DET	PRP\$	소유격 대명사	PUNCT	NFP	불필요한 문장 부호
DET	WDT	관계 대명사	SYM	\$	통화
DET	WP\$	소유격 관계 대명사	VERB	VB	동사 기본형
INTJ	UH	감탄사	VERB	VBD	과거형 동사
NOUN	NN	명사	VERB	VBG	동명사, 현재분사
NOUN	NNS	복수형 명사	VERB	VBN	과거분사
PART	POS	소유격 조사	VERB	VBP	동사 현재형
PART	RP	부사적 불변화사	VERB	VBZ	3인칭 단수 동사 현재형
PART	TO	To 부정사	X	ADD	이메일
PRON	EX	존재문(there)	X	FW	외래어
PRON	PRP	인칭 대명사	X	LS	목록
PRON	WP	관계 대명사	X	SYM	기호

하위 단어 토큰화

- 시간이 지날수록 새로운 단어 출현 or 더 이상 사용 X 단어 생김
- 현대 일상 언어 - 맞춤법 or 띄어쓰기 지켜지지 X 경우 발생
- 신조어 or 고유어 빈번히 발생

⇒ 현대 자연어 처리에는 신조어의 발생, 오타자, 축약어 등을 고려해야 하므로 분석할 단어의 양 증가!

⇒ 하위 토큰화(Subword Tokenization) 출현!

: 하나의 단어가 빈번히 사용되는 하위 단어(Subword)의 조합으로 나누어 토큰화

→ 단어 길이 짧아짐 → 처리 속도 증가

→ OOV 문제, 신조어, 은어, 고유어 등의 문제 완화

- 바이트 페어 인코딩, 워드피스, 유니그램 모델 등

바이트 페어 인코딩(Byte Pair Encoding, BPE)

- (다이어그램 코딩) 하위 단어 토큰화의 한 종류
- 텍스트 데이터에서 가장 빈번히 등장하는 글자 쌍의 조합을 찾아 부호화하는 압축 알고리즘
- 최근에는 자연어 처리 분야에서 하위 단어 토큰화를 위한 방법으로 사용됨
- 연속된 글자 쌍이 더 이상 나타나지 X, 정해진 어휘 사전 크기에 도달할 때까지 조합 탐지, 부호화를 반복

→ 자주 등장하는 단어 : 하나의 토큰으로 토큰화

→ 덜 등장하는 단어 : 여러 토큰의 조합으로 표현

- 센텐스피스(Sentencepiece) & 코포라(Korpora) 라이브러리
- SentencePiece Trainer 매개변수

매개변수	의미
input	말뭉치 텍스트 파일의 경로
model_prefix	모델 파일 이름
vocab_size	어휘 사전 크기
character_coverage	말뭉치 내에 존재하는 글자 중 토큰라이저가 다룰 수 있는 글자의 비율

model_type	토크나이저 알고리즘 ■ unigram ■ bpe ■ char ■ word
max_sentence_length	최대 문장 길이
unk_id	어휘 사전에 없는 OOV를 의미하는 unk 토큰의 id (기본값: 0)
bos_id	문장이 시작되는 지점을 의미하는 bos 토큰의 id (기본값: 1)
eos_id	문장이 끝나는 지점을 의미하는 eos 토큰의 id (기본값: 2)

워드피스(Wordpiece)

- 빈도 기반 X, 확률 기반으로 글자 쌍 병합
- 새로운 하위 단어 생성 시 이전 하위 단어와 함께 나타날 확률을 계산해 가장 높은 확률을 가진 하위 단어를 선택
- 글자 쌍 병합 점수 수식

$$score = \frac{f(x,y)}{f(x)f(y)}$$

[토크나이저스]

- 정규화(Normalization) & 사전 토큰화(Pre-tokenization)
- 정규화 클래스

이름	설명
NFD()	NFD 유니코드 정규화
NFKD()	NFKD 유니코드 정규화
NFC()	NFC 유니코드 정규화
NFKC()	NFKC 유니코드 정규화
Lowercase()	입력 문장의 모든 대문자를 소문자로 변환
Strip()	입력 문장의 양 끝에 있는 공백 제거
StripAccents()	모든 액센트 기호 제거
Replace()	문자 혹은 정규 표현식을 기반으로 입력 문장 변환
BertNormalizer()	BERT 모델에 사용된 정규화 적용
Sequence()	여러 개의 정규화 모듈을 병합해 순서대로 수행

- 사전 토큰화 클래스

클래스	설명
Sequence()	여러 개의 사전 토큰화 모듈을 병합하여 순서대로 수행
ByteLevel()	OpenAI에서 GPT-2를 학습할 때 사용한 방법으로 문자열을 그래픽 문자열로 매핑하고 공백을 기준으로 나눈다.
CharDelimiterSplit()	문자 기준으로 문자열을 나눈다.
Digits()	모든 형태의 숫자 표현을 기준으로 문자열을 나눈다.
Punctuation()	구두점을 기준으로 입력 문자열을 나눈다.
Metaspace(replacement=" ")	Whitespace 기준으로 사전 토큰화를 진행하고, replacement로 Whitespace를 치환한다. * 기본값: _(\u+2581)
Whitespace()	단어 경계공백과 구두점을 기준으로 사전 토큰화를 진행한다. * \w+ [\~\w\s]+ 정규 표현식을 적용한다.
WhitespaceSplit()	공백 문자를 기준으로 입력 문자열을 나눈다.
Split(pattern, behavior)	정규표현식(pattern)과 동작(behavior)을 기준으로 문자열을 나눈다. * behavior - removed: 삭제 - isolated: 개별 토큰 처리 - merged_with_previous: 이전 문자열과 결합해 토큰 처리 - merged_with_next: 다음 문자열과 결합해 토큰 처리 - contiguous: 다음 문자열이 토큰화되지 않아도 결합해 토큰 처리

06 임베딩

- 컴퓨터 - 텍스트 자체를 이해할 수 X! → 텍스트 벡터화 필요!

• 텍스트 벡터화(Text Vectorization)

- 텍스트를 숫자로 변환

1. 원-핫 인코딩(One-Hot Encoding)

- 각 단어를 고유한 색인 값으로 매핑 → 해당 색인 위치를 1로 표시, 나머지 위치는 모두 0으로 표시

2. 빈도 벡터화(Count Vectorization)

- 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표시
- 단어나 문장을 벡터 형태로 변환하기 쉽고 간단
- But, 벡터의 희소성이 크다!
- 컴퓨팅 비용의 증가, 차원의 저주 문제

⇒ 워드 임베딩(Word Embedding) 기법 사용!

Ex. 워드 투 벡터(Word2Vec), 패스트 텍스트(fastText)

- 단어를 고정된 길이의 실수 벡터로 표현하는 방법
- 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현 → 단어 간의 관계를 추론
- But, 고정된 임베딩 학습 → 다의어, 문맥 정보 다루기 어려움

⇒ 동적 임베딩(Dynamic Embedding) 기법 사용

언어 모델(Language Model)

- 입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델
- 주어진 문장을 기반으로 문맥 이해 & 문장 구성 예측 수행
- 자동 번역, 음성 인식, 텍스트 요약 등의 분야에 활용

$P(\text{만나서 반갑습니다.}) = 0.081$	$P(\text{서울특별시}) = 0.59$
Input : 안녕하세요 ...	Input : 대한민국 ... 강남구 ...
$P(\text{아니오. 괜찮습니다.}) = 0.000001$	$P(\text{아이스 아메리카노}) = 0.000001$

- But, 완성된 문장 단위로 확률 계산하는 것은 불가능

⇒ 자기회귀 언어 모델 출현!

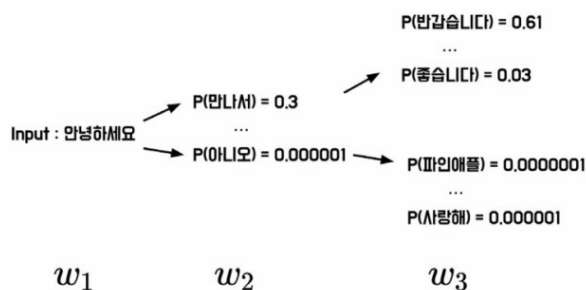
자기회귀 언어 모델(Autoregressive Language Model)

- 입력된 문장들의 조건부 확률을 이용해 다음에 올 단어 예측
- 이전 단어들의 시퀀스 주어짐 → 다음 단어의 확률 계산

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = \frac{P(w_1, w_2, \dots, w_t)}{P(w_1, w_2, \dots, w_{t-1})}$$

- 위의 수식에 조건부 확률의 연쇄법칙(Chain rule for conditional probability) 적용한 수식

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = P(w_1)P(w_2 | w_1) \dots P(w_t | w_1, w_2, \dots, w_{t-1})$$



통계적 언어 모델(Statistical Language Model)

- 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성, 분석
- 시퀀스에 대한 확률 분포를 추정 → 문장의 문맥 파악 → 다음에 등장할 단어의 확률 예측
- **마르코프 체인(Markov Chain)**을 이용해 구현
 - 빈도 기반의 조건부 확률 모델
 - 이전 상태와 현재 상태 간의 전이 확률 이용 → 다음 상태 예측
 - But, 단어의 순서, 빈도에만 기초해 문장의 확률을 예측 → 불완전 or 부적절한 결과 생성 가능
 - 한 번도 등장 X 단어나 문장 → 정확한 예측이 어려움 ⇒ 데이터 희소성 문제!
- **통계적 언어 모델**
 - 새로운 문장 생성 가능 & 다양한 종류의 텍스트 데이터 학습 가능
 - 대규모 자연어 데이터 처리에 효과적
 - 강력한 모델 구현 가능
- 트랜스포머 모델에 기반한 언어 모델인 GPT(Generative Pre-Trained Transformer) & BERT(Bidirectional Encoder Representations from Transformers) → 언어 모델을 활용해 가중치 사전 학습

GPT

- Raw Sentence: GPT는 OpenAI에서 개발한 인과적 언어 모델입니다.
- Input: GPT는 OpenAI에서 개발한
- Prediction-1: 인과적
- Prediction-2: 언어 모델입니다.

BERT

- Raw Sentence: BERT는 Google에서 개발한 마스킹된 언어 모델입니다.
- Input: Bert는 [MASK]에서 개발한 [MASK] 언어 모델입니다.
- Prediction: Google, 마스킹된

N-gram

- 가장 기초적인 통계적 언어 모델
- N개의 연속된 단어 시퀀스를 하나의 단위로 취급 → 특정 단어 시퀀스가 등장할 확률을 추정
- 입력 텍스트를 N개의 토큰으로 묶어서 분석

Unigram : 안녕하세요 만나서 진심으로 반가워요 안녕하세요, 만나서, 진심으로, 반가워요
 Bigram : 안녕하세요 만나서 진심으로 반가워요 안녕하세요 만나서, 만나서 진심으로, 진심으로 반가워요
 Trigram : 안녕하세요 만나서 진심으로 반가워요 안녕하세요 만나서 진심으로, 만나서 진심으로 반가워요

- N-1개 토큰만을 고려해 확률을 계산

$$P(w_l | w_{l-1}, w_{l-2}, \dots, w_{l-N+1})$$

- 단어의 순서가 중요한 자연어 처리 작업 및 문자열 패턴 분석에 활용

TF-IDF(Term Frequency-Inverse Document Frequency)

- 특정 단어의 중요도를 계산하는 방법
- 단어의 중요도를 평가하는 데 사용되는 통계적인 가중치
- **BoW(Bag-of-Words)**에 가중치를 부여하는 방법
 - 문서나 문장을 단어의 집합으로 표현하는 방법
 - 단어의 중복을 허용해 빈도를 기록
 - 모든 단어는 동일한 가중치

단어 빈도(Term Frequency, TF)

- 문서 내에서 특정 단어의 빈도수를 나타내는 값

$$TF(t, d) = count(t, d)$$

- 문서의 길이가 길어질수록 해당 단어의 TF 값도 높아질 수 O!

문서 빈도(Document Frequency, DF)

- 한 단어가 얼마나 많은 문서에 나타나는가
- 문서 집합에서 단어가 나타나는 횟수 계산

$$DF(t, D) = count(t \in d : d \in D)$$

- 값이 높으면 → 일반적으로 널리 사용, 중요도 낮을 수 O!

- 값이 낮으면 → 특정 문맥에서만 사용될 가능성 → 중요도 높을 수 O!

역문서 빈도(Inverse Document Frequency, IDF)

- 전체 문서 수를 문서 빈도로 나눈 후 로그를 취한 값
- 문서 내에서 특정 단어가 얼마나 중요한지 나타냄
- 문서 빈도 높 → 단어가 일반적이고 상대적으로 중요하지 X

→ 문서 빈도의 역수를 취하면 → 단어의 빈도수가 적을수록 → IDF 값이 커지게 보정

$$IDF(t,D) = \log\left(\frac{count(D)}{1 + DF(t,D)}\right)$$

TF-IDF

- 문서 빈도 X 역문서 빈도

$$TF-IDF(t,d,D) = TF(t,d) \times IDF(t,d)$$

- 문서 내에서 단어 자주 등장 & 전체 문서 내에서는 적게 등장 → Tf-IDF 값 증가

⇒ 관사 or 관용어 등의 가중치는 감소